

CPU Scheduling Algorithms I

Week 5: FCFS, SJF/SRTF, priority scheduling, and Round Robin

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. state a scheduling model and all tie-breaking assumptions before solving it;
2. compute completion, turnaround, waiting, response, and slowdown metrics;
3. construct FCFS, SJF, SRTF, priority (both variants), and RR schedules;
4. state precisely when SJF/SRTF optimality results apply;
5. explain convoy effects, starvation, prediction error, and quantum trade-offs;
6. incorporate idle time, context-switch cost, arrivals, and blocking correctly; and
7. generalize size-based selection to explicit priority keys without assuming the same optimality guarantees.

Guiding question

Which schedule is “best”—and best according to whose objective, information, and assumptions?

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

Start with a model, not an algorithm name

A scheduling problem must specify:

Workload assumptions

- one CPU or multiple/heterogeneous CPUs;
- arrival/release time a_i ;
- CPU service/burst b_i ;
- deadlines, weights, priorities;
- CPU-only jobs or alternating CPU/I/O bursts.

Scheduler assumptions

- preemptive or non-preemptive;
- clairvoyant or predicted service times;
- work-conserving or deliberate idling;
- context-switch/preemption cost;
- tie-breaking and arrival-at-boundary rules.

Without these, the answer is not unique

Two correct implementations can produce different Gantt charts when equal keys, simultaneous arrivals, or quantum-boundary arrivals are unspecified.

Conventions for every numerical example

- One CPU; independent jobs with one known CPU burst each; no I/O.
- $P_i = (a_i, b_i)$ gives arrival and burst; priority examples add p_i .
- Time starts at zero. Dispatch/switch cost is zero unless explicitly added.
- FCFS arrival ties: smaller process index first. SJF/SRTF dispatch ties: earlier original arrival, then smaller process index.
- SRTF and preemptive priority retain the current job on an equal key; preempt only for a strictly better key.
- RR enqueues arrivals through the boundary before requeueing the expired job; simultaneous arrivals use process-index order.

Charts use half-open intervals [start, end). Each example repeats its complete workload. Priority tie rules are restated in that section.

Notation and primary latency metrics

For job i :

a_i = arrival, s_i = first start, c_i = completion, b_i = total CPU service.

$T_i = c_i - a_i$ turnaround,
 $R_i = s_i - a_i$ first response,
 $W_i = T_i - b_i$ CPU-only ready wait,
 $S_i = \frac{T_i}{b_i}$ slowdown.

Waiting-time caveat

In a general CPU/I/O workload,

$T_i = \text{CPU} + \text{ready wait} + \text{blocked time}.$

Therefore $T_i - b_i$ includes blocking and is not purely ready-queue waiting.

System-level objectives conflict

Objective	Measure	Potential conflict
throughput	completions per unit time	batching may increase response latency
utilization	fraction of capacity doing defined useful work	100% load can create long queues
mean turnaround	$\frac{1}{n} \sum_i T_i$	can sacrifice long jobs and tail latency
response	mean/percentiles of R_i	frequent preemption harms locality
fairness	equal/weighted service over an interval	may reduce mean completion performance
deadline success	misses, tardiness, lateness	can reject or deprioritize best-effort work
predictability	variance, high percentiles, bounds	conservative control may reduce average throughput
energy	joules/task or performance per watt	consolidation/frequency changes latency

Average metrics can hide unacceptable schedules

Report more than one mean:

- median and high-percentile latency;
- maximum wait over a stated interval;
- per-class/user/group service;
- slowdown for short versus long jobs;
- starvation/deadline violations;
- scheduler and context-switch overhead.

Example

Nine jobs finish immediately and one waits 100 seconds. A modest mean can coexist with an unusable tail and a fairness failure.

Optimization needs a stakeholder

Minimizing total completion time values completed jobs equally, not users, urgency, revenue, or safety.

Online/offline and clairvoyant/non-clairvoyant

Arrival information

Online: only released jobs are known.

Offline: the entire instance, including future arrivals, is known.

- Online does not mean size-blind.
- Offline information can support deliberate idling.

Service-time information

Clairvoyant: knows a released job's service requirement.

Non-clairvoyant: learns through execution or uses estimates.

- Exact-size SRTF can be online *and* clairvoyant.
- Predicted sizes do not provide exact-size guarantees.

Evaluation rule

Do not compare a clairvoyant optimum with an online policy without acknowledging the information advantage.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served**
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

FCFS/FIFO: arrival order

Policy: select the oldest ready job; in the classical version, run until completion or blocking.

- simple FIFO ready queue;
- non-preemptive among CPU bursts;
- deterministic given arrival/tie order;
- no starvation for a finite queue with finite service times;
- insensitive to job size or urgency.

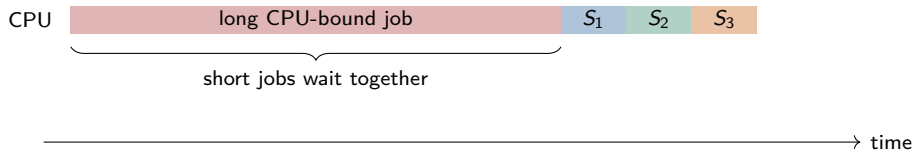
Strength

Low policy overhead and a transparent temporal ordering.

Weakness

A long CPU burst can delay many short or latency-sensitive jobs even when completing them first would greatly reduce total flow time.

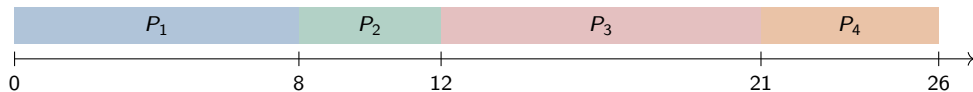
Convoy effect: synchronization of delays



- Several short/I/O-oriented jobs accumulate behind one long CPU burst.
- When released, they may run briefly, issue I/O, and leave the CPU-bound job at the head again.
- CPU utilization need not always be low; the deeper issue is poor overlap, long response, and correlated device/CPU idle periods in some workloads.

FCFS worked example

Workload: $P_1 = (a = 0, b = 8)$, $P_2 = (1, 4)$, $P_3 = (2, 9)$, $P_4 = (3, 5)$.



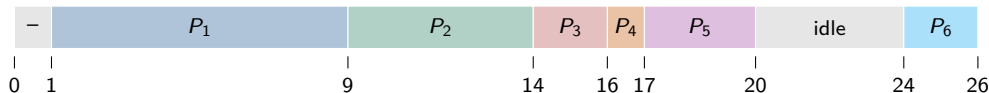
Job	c_i	T_i	W_i	R_i	S_i
P_1	8	8	0	0	1.00
P_2	12	11	7	7	2.75
P_3	21	19	10	10	2.11
P_4	26	23	18	18	4.60
mean		15.25	8.75	8.75	2.62

FCFS: six jobs, tied arrivals, and idle time

Workload $P_i = (a_i, b_i)$:

$$\begin{aligned} P_1 &= (1, 8), & P_2 &= (2, 5), & P_3 &= (3, 2), \\ P_4 &= (5, 1), & P_5 &= (5, 3), & P_6 &= (24, 2). \end{aligned}$$

Zero overhead; no I/O; at the tied arrival, P_4 precedes P_5 .



Gray intervals denote an idle CPU, including $[0, 1)$.

Final means (rounded to two decimals)

$$\bar{T} = 10.33 \quad \bar{W} = 6.83 \quad \bar{R} = 6.83 \quad \bar{S} = 4.65$$

T, W, R : time units; S : dimensionless slowdown.

Idle intervals must appear in the schedule

Jobs: $A = (a = 3, b = 2)$, $B = (a = 8, b = 1)$.



- A work-conserving scheduler idles only when no eligible job is ready.
- Omitting idle time changes completion and response calculations.
- Non-work-conserving schedules can intentionally idle for energy, reservations, or future high-value/deadline work; FCFS normally does not.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time**
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

SJF and SRTF policies

Non-preemptive SJF/SPT

When the CPU becomes free, select the ready job with the smallest predicted/known service time; run its burst to completion or block.

- Both privilege short jobs and can starve long jobs under a continuing short-job stream.
- Exact remaining service is normally unavailable; prediction or application declarations are required.
- Preemption improves the clairvoyant objective but adds switch/locality costs ignored by the ideal model.

Preemptive SRTF/SRPT

Always run the available job with the least remaining service; a shorter arrival can preempt the current job.

What exactly is SJF optimal for?

Precise statement

For jobs available together on one identical machine, with known processing times, no preemption, and no overhead, ordering by nondecreasing processing time minimizes $\sum_i c_i$ and therefore mean completion/turnaround time.

- With release times, unknown sizes, weights, deadlines, blocking, or multiple CPUs, the statement requires modification and may be false.
- If deliberate idling is allowed and future arrivals are known, immediately running the shortest currently ready job need not be globally optimal.
- SRTF/SRPT provides a corresponding preemptive optimum for total flow time under its ideal single-machine clairvoyant assumptions.

Exchange proof for SJF optimality

Consider adjacent jobs i then j beginning at time t , with $b_i > b_j$.

Original order i, j contributes

$$(t + b_i) + (t + b_i + b_j) = 2t + 2b_i + b_j.$$

Difference:

$$(2t + 2b_i + b_j) - (2t + 2b_j + b_i) = b_i - b_j > 0.$$

Swapped order j, i contributes

$$(t + b_j) + (t + b_j + b_i) = 2t + 2b_j + b_i.$$

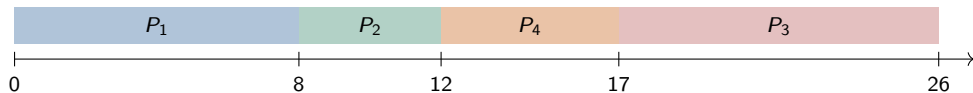
Thus swapping an inversion reduces total completion time. Repeatedly remove inversions to obtain shortest-processing-time order.

Proof boundary

The argument assumes both jobs are available at t , their processing times are known, and swapping does not affect other constraints.

Non-preemptive SJF worked example

Workload (a_i, b_i): $P_1 = (0, 8)$, $P_2 = (1, 4)$, $P_3 = (2, 9)$, $P_4 = (3, 5)$.



Job	c_i	T_i	W_i	R_i
P_1	8	8	0	0
P_2	12	11	7	7
P_3	26	24	15	15
P_4	17	14	9	9
mean		14.25	7.75	7.75

At $t = 0$, only P_1 is available. Non-preemptive SJF cannot replace it when shorter jobs arrive.

SJF: six jobs and non-preemptive choices

Workload $P_i = (a_i, b_i)$:

$$\begin{aligned} P_1 &= (1, 8), & P_2 &= (2, 5), & P_3 &= (3, 2), \\ P_4 &= (5, 1), & P_5 &= (5, 3), & P_6 &= (24, 2). \end{aligned}$$

Zero overhead; no I/O; choose shortest only when the CPU becomes free.



Gray intervals denote an idle CPU, including $[0, 1)$.

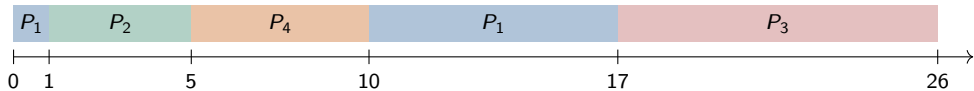
Final means (rounded to two decimals)

$$\overline{T} = 8.67 \quad \overline{W} = 5.17 \quad \overline{R} = 5.17 \quad \overline{S} = 3.07$$

T, W, R : time units; S : dimensionless slowdown.

SRTF worked example

Workload (a_i, b_i) : $P_1 = (0, 8)$, $P_2 = (1, 4)$, $P_3 = (2, 9)$, $P_4 = (3, 5)$.



Job	c_i	T_i	W_i	R_i
P_1	17	17	9	0
P_2	5	4	0	0
P_3	26	24	15	15
P_4	10	7	2	2
mean		13.00	6.50	4.25

Why SRTF did not preempt at every arrival

Workload (a_i, b_i) : $P_1 = (0, 8)$, $P_2 = (1, 4)$, $P_3 = (2, 9)$, $P_4 = (3, 5)$.

Track remaining service:

- $t = 1$: P_1 has 7 remaining; P_2 arrives with 4, so preempt.
- $t = 2$: P_2 has 3 remaining; P_3 arrives with 9, so keep P_2 .
- $t = 3$: P_2 has 2 remaining; P_4 arrives with 5, so keep P_2 .
- $t = 5$: among $P_1(7)$, $P_3(9)$, $P_4(5)$, select P_4 .

Common error

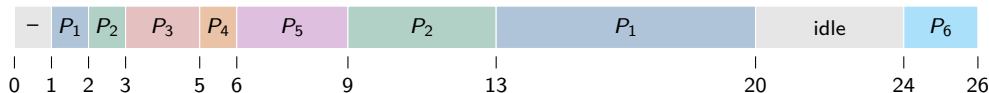
“A new job arrived” is not itself a reason to preempt. Preempt only if the policy’s ordering key makes the new/other job preferable after applying tie rules and switching cost assumptions.

SRTF: nested preemptions and simultaneous arrivals

Workload $P_i = (a_i, b_i)$:

$$\begin{aligned} P_1 &= (1, 8), & P_2 &= (2, 5), & P_3 &= (3, 2), \\ P_4 &= (5, 1), & P_5 &= (5, 3), & P_6 &= (24, 2). \end{aligned}$$

Zero overhead; no I/O; process all arrivals before the next selection.



Gray intervals denote an idle CPU, including $[0, 1)$.

Final means (rounded to two decimals)

$$\overline{T} = 6.50 \quad \overline{W} = 3.00 \quad \overline{R} = 0.17 \quad \overline{S} = 1.48$$

T, W, R : time units; S : dimensionless slowdown.

Predicting the next CPU burst

A classical exponential predictor is

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n, \quad 0 \leq \alpha \leq 1,$$

where t_n is the observed burst and τ_n the previous prediction.

Larger α :

- reacts quickly to phase changes;
- increases noise sensitivity;
- forgets older history rapidly.

Smaller α :

- smooths noise;
- adapts slowly;
- can lag changing workloads.

Prediction changes the policy

Shortest *predicted* job is not clairvoyant SJF. Prediction errors can reverse priorities, cause starvation, and be gamed by strategic workloads.

Starvation and aging

SJF/SRTF can postpone a long job indefinitely if short jobs keep arriving faster than the long job receives service.

- finite initial job sets eventually finish;
- open systems need an explicit liveness/fairness policy;
- tail latency can be poor despite an excellent mean.

Aging

Increase effective priority as waiting grows, or blend size with wait:

$$\text{score}_i = f(\hat{b}_i, W_i, w_i).$$

Trade-off

Stronger aging protects liveness but moves the schedule away from pure shortest-job optimality.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling**
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

From SRTF to a general priority key

SRTF succeeds for its idealized mean-flow objective by giving preference to the least remaining work. Generalize the *selection mechanism*:

$$\text{next}(t) \in \arg \min_{i \in \mathcal{R}(t)} k_i(t).$$

Here $\mathcal{R}(t)$ contains all runnable jobs, including the current job.

Policy	Key (smaller is better)	Reconsider execution
Non-preemptive SJF	$k_i = b_i$	when CPU becomes free
SRTF	$k_i = r_i(t)$, remaining service	at arrivals/completions
Fixed priority	$k_i = p_i$, assigned rank	according to preemption rule

Generalizing the key does not preserve the theorem

An arbitrary priority ranks importance, not remaining work. It does not inherit SRTF's minimum mean-flow guarantee or promise deadlines.

Priority scheduling: two execution rules

Workload notation: $P_i = (a_i, b_i, p_i)$.

Convention for this deck

Smaller number means higher priority: $p = 1$ outranks $p = 2$. This numerical direction is a convention, not a universal OS rule.

Non-preemptive priority

At dispatch, choose the ready job with minimum p_i . It keeps the CPU until its burst ends or it blocks.

Preemptive priority

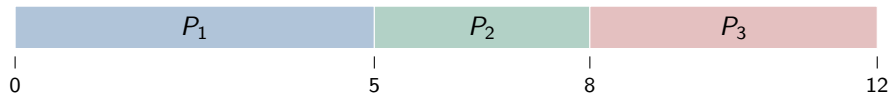
A newly runnable job with strictly smaller p_i displaces the running job; preserve the displaced job's remaining service.

Equal-priority dispatch ties: **earlier original arrival**, then process index. Equal priority alone never preempts the running job. No equal-priority time slicing here.

Non-preemptive priority: worked example

Workload (a_i, b_i, p_i) : $P_1 = (0, 5, 3)$, $P_2 = (1, 3, 1)$, $P_3 = (2, 4, 2)$.

Lower p_i is higher priority; zero overhead; no I/O.



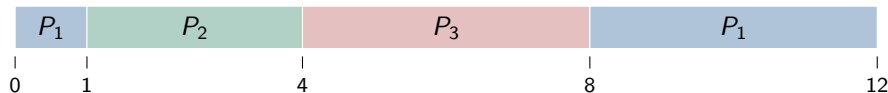
Job	c_i	T_i	W_i	R_i
P_1	5	5	0	0
P_2	8	7	4	4
P_3	12	10	6	6
mean		7.33	3.33	3.33

P_2 is highest priority but cannot displace P_1 at $t = 1$. Priority is consulted only at dispatch.

Preemptive priority: worked example

Workload (a_i, b_i, p_i) : $P_1 = (0, 5, 3)$, $P_2 = (1, 3, 1)$, $P_3 = (2, 4, 2)$.

Lower p_i is higher priority; zero overhead; no I/O.



Job	c_i	T_i	W_i	R_i
P_1	12	12	7	0
P_2	4	3	0	0
P_3	8	6	2	2
mean		7.00	3.00	0.67

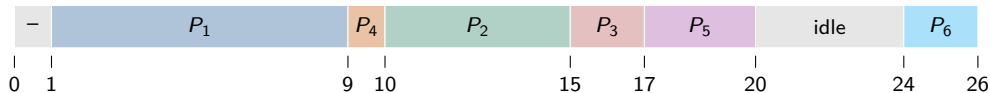
At $t = 1$, P_2 preempts P_1 . At $t = 2$, P_3 does not preempt P_2 : 2 is worse than 1.

Non-preemptive priority: six jobs and tied ranks

Workload $P_i = (a_i, b_i, p_i)$:

$$\begin{aligned} P_1 &= (1, 8, 4), & P_2 &= (2, 5, 2), & P_3 &= (3, 2, 2), \\ P_4 &= (5, 1, 1), & P_5 &= (5, 3, 3), & P_6 &= (24, 2, 1). \end{aligned}$$

Zero overhead; no I/O; smaller p wins, then earlier arrival/index.



Gray intervals denote an idle CPU, including $[0, 1)$.

Final means (rounded to two decimals)

$$\overline{T} = 9.50 \quad \overline{W} = 6.00 \quad \overline{R} = 6.00 \quad \overline{S} = 3.60$$

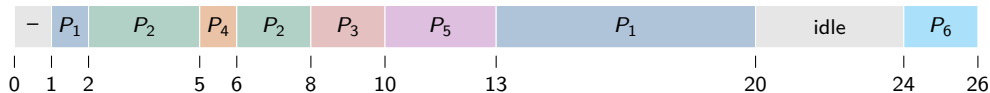
T, W, R : time units; S : dimensionless slowdown.

Preemptive priority: interruptions and equal ranks

Workload $P_i = (a_i, b_i, p_i)$:

$$\begin{aligned} P_1 &= (1, 8, 4), & P_2 &= (2, 5, 2), & P_3 &= (3, 2, 2), \\ P_4 &= (5, 1, 1), & P_5 &= (5, 3, 3), & P_6 &= (24, 2, 1). \end{aligned}$$

Zero overhead; no I/O; strictly better p preempts; ties retain current job.



Equal-priority dispatch ties: original arrival, then index. Gray means idle.

Final means (rounded to two decimals)

$$\overline{T} = 7.17 \quad \overline{W} = 3.67 \quad \overline{R} = 1.67 \quad \overline{S} = 1.96$$

T, W, R : time units; S : dimensionless slowdown.

Priority: starvation, aging, and its limits

Strict fixed priority can indefinitely postpone a ready low-priority job under a continuing higher-priority workload.

One explicit aging rule (smaller is better)

$$p_i^{\text{eff}}(t) = \max \left(p_{\min}, p_i^{\text{base}} - \left\lfloor \frac{w_i^{\text{ready}}(t)}{\Delta} \right\rfloor \right).$$

w_i^{ready} : cumulative ready wait since the most recent dispatch, reset at dispatch. Every Δ waiting units improve the rank by one.

- Reevaluate ordering when an aging threshold is crossed, not just at arrivals.
- Reaching the top rank is insufficient if equal-rank jobs can barge ahead or run forever; specify fair ties and finite service.
- Aging changes the objective: low base priority no longer means permanent subordination. It need not minimize mean turnaround.

Priority policies compared with size-based policies

Question	Non-preemptive priority	Preemptive priority
Higher-priority arrival?	waits for current burst	displaces lower-priority job
Equal-priority arrival?	arrival/index tie at dispatch	no preemption; arrival/index tie at dispatch
Needs burst prediction?	no	no
Mean-flow optimality?	no general guarantee	no general guarantee
Starvation risk?	sustained higher-priority work	sustained higher-priority work

Keep policy and objective separate

SJF/SRTF rank size to improve a size-sensitive objective. Fixed priority ranks assigned importance. RR, next, instead rotates service among ready peers.

Multilevel queues, feedback variants, and real-time algorithms are reserved for later lecture sets.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin**
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 Synthesis

Round Robin: bounded turns in FIFO order

Policy: give the head task up to quantum q ; if still runnable after q , preempt and append it to the tail.

- New arrivals join the ready queue according to a specified boundary rule.
- A task that blocks before q relinquishes the CPU; unused quantum is normally not saved.
- For a fixed set of n continuously runnable peers and zero overhead, end-of-turn to next-start wait is at most $(n - 1)q$; start-to-start spacing is at most nq .
- RR provides turn-taking, not universal fairness across weights, priorities, blocking patterns, or multicore placement.

Limiting behavior

$q \rightarrow \infty$ approaches FCFS. Very small q approaches processor sharing in service allocation but incurs high switching and locality cost.

RR queue semantics at a quantum boundary

Workload (a, b) : $A = (0, 4)$, $B = (2, 2)$; $q = 2$, zero overhead.

At $t = 2$, A 's quantum expires exactly when B arrives.

Rule 1

Process arrivals first, then append expired
 A : queue becomes B, A .

Rule 2

Append expired A first, then process arrival:
queue becomes A, B .

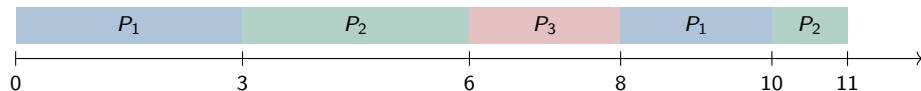
Both can be internally consistent

The Gantt chart and response times differ. State the event-ordering convention before solving or simulating RR.

This deck processes arrivals through time t before requeueing the just-expired task at t .

RR worked example, $q = 3$

Workload (a_i, b_i) : $P_1 = (0, 5)$, $P_2 = (1, 4)$, $P_3 = (2, 2)$.



Job	c_i	T_i	W_i	R_i
P_1	10	10	5	0
P_2	11	10	6	2
P_3	8	6	4	4
mean		8.67	5.00	2.00

RR state table prevents Gantt-chart mistakes

Workload (a_i, b_i) : $P_1 = (0, 5)$, $P_2 = (1, 4)$, $P_3 = (2, 2)$.

$q = 3$; zero overhead; arrivals enqueue before an expired job is requeued.

Time	Ready queue before dispatch	Run	Remaining afterward
0	$[P_1]$	$P_1 : 0 \rightarrow 3$	$P_1 = 2$
3	$[P_2, P_3, P_1]$	$P_2 : 3 \rightarrow 6$	$P_2 = 1$
6	$[P_3, P_1, P_2]$	$P_3 : 6 \rightarrow 8$	completes
8	$[P_1, P_2]$	$P_1 : 8 \rightarrow 10$	completes
10	$[P_2]$	$P_2 : 10 \rightarrow 11$	completes

Method

For RR: run until completion, blocking, or quantum expiry. Arrivals enqueue without preempting. At the boundary, enqueue arrivals before appending an unfinished expired job.

RR: six jobs, boundary arrivals, and idle time

Workload $P_i = (a_i, b_i)$:

$$\begin{aligned} P_1 &= (1, 8), & P_2 &= (2, 5), & P_3 &= (4, 2), \\ P_4 &= (5, 1), & P_5 &= (5, 3), & P_6 &= (24, 2). \end{aligned}$$

$q = 3$; zero overhead; no I/O; arrivals enqueue before the expired job.



Simultaneous arrivals use process-index order. Gray means idle.

Final means (rounded to two decimals)

$$\bar{T} = 10.17 \quad \bar{W} = 6.67 \quad \bar{R} = 3.33 \quad \bar{S} = 3.46$$

T, W, R : time units; S : dimensionless slowdown.

Choosing the quantum is workload-dependent

Small q :

- improves turn frequency;
- approximates fluid sharing;
- raises direct switch overhead;
- harms cache/TLB locality;
- can fragment useful bursts.

Large q :

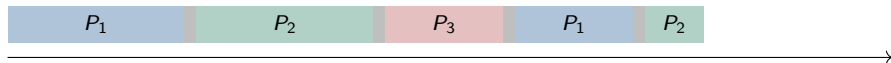
- improves locality/throughput;
- reduces scheduler overhead;
- approaches FCFS;
- increases worst-case turn wait;
- can hurt interactive response under load.

No universal 80% rule

A common heuristic is to choose q so many interactive CPU bursts finish within one slice, but the correct value depends on workload, switch cost, latency goals, CPU count, and scheduler design.

Include context-switch cost in the schedule

Workload (a_i, b_i) : $P_1 = (0, 5)$, $P_2 = (1, 4)$, $P_3 = (2, 2)$; RR $q = 3$.
Different-task handoffs cost $s = 0.2$; initial dispatch is free; no I/O.



- Completion times shift; recompute every metric from the cost-aware timeline.
- Whether idle-to-task or same-task redispatch costs s must also be specified.
- SRTF can stop being attractive when predicted savings are smaller than preemption/warmup cost.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights**
- 7 Simulation and Verification
- 8 Synthesis

FCFS, SJF, SRTF, and RR: comparison

	FCFS	SJF	SRTF	RR
preemption	no	no	shorter remainder	quantum expiry
ranking input	arrival order	burst size	remaining service	FIFO turn order
ideal mean flow	may be poor	optimal if all ready together	preemptive optimum	depends on workload and q
first response	long-burst delays	short-job preference	short-arrival preference	bounded turn wait*
starvation	none*	long jobs at risk	long jobs at risk	none*
main risk	convoy	prediction error	switch cost	quantum overhead
ready structure	FIFO	size-keyed heap	remainder-keyed heap	FIFO plus timer

Exact positive sizes, independent CPU-only jobs, one CPU, zero overhead. *Finite ready set, finite bursts, and one service class. SJF/SRTF starvation concerns continuing arrivals.

Different objectives favor different policies

A scheduler can be preferable under different objectives:

- **Mean turnaround:** clairvoyant SRTF often wins in the ideal model.
- **Temporal transparency:** FCFS is easy to explain and reproduce.
- **Turn frequency/interactive sharing:** RR prevents one long CPU burst from holding the CPU indefinitely.
- **Tail protection:** none of these alone handles priorities, weights, or bounded starvation fully.
- **Production efficiency:** prediction accuracy, cache behavior, multicore placement, and overhead can reverse the textbook ranking.

No universal champion

An algorithm is optimal only relative to an objective, information model, constraints, and workload distribution.

Competitive thinking for online schedulers

An online scheduler is evaluated against an offline optimum that knows the future:

$$\text{competitive ratio} = \sup_{\text{workloads}} \frac{\text{cost of online algorithm}}{\text{cost of optimal offline algorithm}}.$$

- This exposes how lack of future knowledge limits guarantees.
- Adversarial arrivals can make simple policies perform arbitrarily badly for some objectives.
- Resource augmentation asks whether a slightly faster online machine can compete with an optimal baseline.
- Average-case traces and worst-case competitive analysis answer different questions.

A full competitive analysis is beyond this lecture, but the framework explains why clairvoyant SJF is an unfair production baseline unless its information advantage is explicit.

Multicore scheduling breaks the single timeline

Additional decisions:

- which CPU executes each thread;
- global versus per-CPU queues;
- affinity/cache/NUMA preservation;
- load balancing and migration;
- co-runner interference and SMT;
- heterogeneous CPU capacity/energy.

Proofs do not transfer automatically

SJF/SRTF single-machine optimality does not directly solve parallel-machine assignment, migration, precedence, or cache-affinity constraints.

Visual model

Use one timeline per CPU plus migration arrows and explicit shared-resource constraints.

Fairness requires a definition

Possible definitions:

- equal turns or equal CPU time per task;
- weighted proportional share;
- max-min fairness;
- bounded lag from an ideal fluid server;
- per-user/cgroup rather than per-thread fairness;
- priority- or deadline-constrained service.

RR is not automatically fair

Tasks that block, have different priorities, run on different CPUs, or belong to different weights receive different service. Equal quantum is only one local mechanism.

Production view

Fair schedulers often approximate a weighted fluid allocation while retaining efficient discrete dispatch.

Real systems combine mechanisms

- General-purpose kernels use multiple scheduling classes/policies, priorities, weights, CPU affinity, and load balancing.
- Feedback mechanisms infer interactivity/CPU demand rather than know future bursts.
- Real-time classes use fixed-priority or deadline-oriented policies with admission/priority rules.
- User-space runtimes schedule tasks onto kernel threads; Grand Central Dispatch is a concurrency runtime/framework, not the kernel CPU scheduler itself.
- Containers/cgroups introduce hierarchical shares and quotas above per-thread decisions.

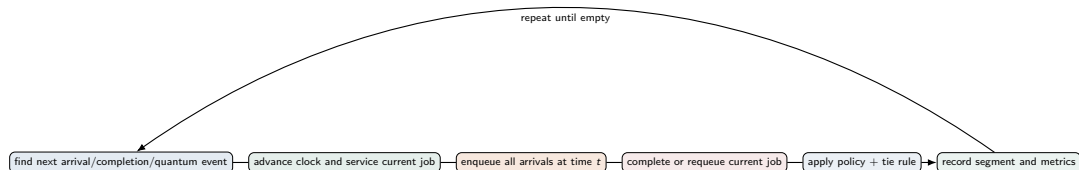
Avoid stale implementation shorthand

Do not equate all Linux fair scheduling with one historic data structure or all Windows behavior with a fixed version-specific number of levels. Inspect the target kernel/version.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification**
- 8 Synthesis

Event-driven simulation structure



- Event-driven simulation avoids iterating through every time unit.
- Maintain remaining service, first-start time, completion time, and ready-queue order explicitly.
- Assert invariants: no execution before arrival, one CPU at a time, and total service equals b_i .

Policy-independent metric calculation

```
1 for job in jobs:
2     turnaround = job.completion - job.arrival
3     response   = job.first_start - job.arrival
4     waiting    = turnaround - job.service # CPU-only model
5     assert turnaround >= job.service
6     assert response >= 0
```

- Calculate metrics from recorded timestamps, not by visually guessing the chart.
- For CPU/I/O traces, accumulate ready-queue residence directly rather than using $T_i - b_i$.
- Preserve exact rational values until presentation; round only final reported statistics.

Verification checklist for a Gantt chart

1. Every job receives exactly its required total service.
2. No job executes before arrival or while modeled as blocked.
3. At each dispatch, the chosen job is optimal under the stated policy and tie rule.
4. Every preemption is triggered by a valid policy event.
5. Idle and context-switch intervals are explicit.
6. Completion is the end of the final segment; first start is the beginning of the first.
7. Mean metrics agree with per-job values and units.

Road map

- 1 Model and Evaluation Metrics
- 2 First-Come, First-Served
- 3 Shortest-Job and Shortest-Remaining-Time
- 4 Priority Scheduling
- 5 Round Robin
- 6 Comparison and Deeper Insights
- 7 Simulation and Verification
- 8 **Synthesis**

Misconceptions corrected

Misconception	⇒	Precise statement
SJF is always optimal.	⇒	Its optimality is objective- and model-specific and assumes size knowledge.
$W = T - b$ always measures ready waiting.	⇒	Only in the CPU-only model; blocking must otherwise be separated.
FCFS guarantees good fairness.	⇒	It preserves arrival order but can create extreme size-dependent delay.
Every arrival preempts under SRTF.	⇒	Preempt only if remaining-time ordering changes.
RR eliminates starvation universally.	⇒	Only under finite equal-class runnable assumptions; priorities/weights can differ.
A fixed 80% burst heuristic gives the right quantum.	⇒	Quantum selection is workload- and system-dependent.
RR is the modern desktop scheduler.	⇒	Production schedulers combine fair-share, priority, topology, and other mechanisms.
Grand Central Dispatch schedules CPUs.	⇒	It schedules application work onto threads; the kernel schedules those threads.

Key takeaways

- A schedule is meaningful only with explicit workload, information, overhead, and tie assumptions.
- FCFS is transparent but vulnerable to convoys and poor short-job latency.
- SJF/SRTF optimize idealized mean flow objectives using exact service-size information that real workloads rarely provide.
- Explicit priority generalizes the ranking key, not SRTF's optimality guarantee.
- Prediction makes shortest-job policies practical but fallible and potentially unfair.
- RR trades locality and overhead for bounded turn frequency under a defined runnable set.
- Mean metrics, tail latency, slowdown, fairness, and deadlines expose different quality dimensions.
- Single-CPU textbook results are foundations, not complete multicore production designs.

Laboratory: build a scheduling simulator

Implement FCFS, SJF, SRTF, both priority variants, and RR with:

1. event-driven arrivals and explicit idle intervals;
2. explicit priority direction, tie-breaking, and RR boundary ordering;
3. optional dispatch/preemption cost;
4. per-job c_i , T_i , W_i , R_i , S_i and aggregate percentiles;
5. invariant checks and a generated Gantt chart;
6. random and adversarial workload generators.

Experiment

Sweep quantum and prediction α ; compare mean flow time, $p95$ response, switch count, and starvation/maximum wait.

Later lecture sets: queues, feedback, and real time

- multilevel queues and inter-queue service policies;
- multilevel feedback queues and their variations;
- proportional-share and lottery/stride scheduling;
- real-time algorithms and schedulability analysis;
- production scheduling classes and implementation case studies.

Preparation

Today's core: FCFS, SJF, SRTF, non-preemptive/preemptive priority, and RR. Explain which ranking key and preemption rule each uses before adding multiple queues or deadlines.

Road map

9 Appendix

Knowledge check: metrics and assumptions

1. Why is $W_i = T_i - b_i$ invalid when a task blocks for I/O?
2. What tie rules are needed to make an RR answer reproducible?
3. Give a workload where mean turnaround and fairness prefer different schedules.
4. Why is response-time terminology ambiguous across scheduling fields?
5. What information advantage does clairvoyant SRTF possess?
6. Why must context-switch intervals be included before recomputing metrics?

Knowledge check: algorithms

1. State the assumptions behind non-preemptive SJF's exchange proof.
2. Can FCFS starve a job in a finite queue with finite bursts?
3. Why can SRTF starve a long job in an open system?
4. When does RR approach FCFS?
5. Does equal quantum imply equal CPU share for tasks that frequently block?
6. Why does SRTF not preempt on every arrival?

Knowledge check: priority is a policy, not a guarantee

1. In what sense are SJF and SRTF special cases of priority-based selection?
2. Why does substituting assigned ranks for remaining time lose SRTF's optimality guarantee?
3. What happens when a higher-priority job arrives during a non-preemptive burst?
4. Can a shorter job of equal priority preempt under this deck's priority rules?
5. Which events should trigger reevaluation if priority changes through aging?
6. What assumptions are needed before claiming that aging prevents starvation?

Exercise: solve the common workload

Jobs: $P_1 = (0, 8)$, $P_2 = (1, 4)$, $P_3 = (2, 9)$, $P_4 = (3, 5)$.

1. Construct FCFS, non-preemptive SJF, and SRTF schedules.
2. Compute c_i , T_i , W_i , R_i , S_i for every job.
3. Add a 0.5 unit cost for each handoff to a different job.
4. Does the relative ranking by mean turnaround change?
5. Compare maximum wait and maximum slowdown.

Exercise: RR boundary rules

Jobs: $A = (0, 4)$ and $B = (2, 2)$ with $q = 2$.

1. Draw the schedule if arrivals at $t = 2$ enqueue before expired A .
2. Draw it if expired A requeues before arrivals at $t = 2$.
3. Compare response and completion times.
4. Which rule does your simulator implement, and where is it documented?

Lesson

A tiny event-ordering detail changes the answer even though both implementations are recognizably Round Robin.

Exercise: burst prediction

Let initial prediction $\tau_0 = 6$ and observed bursts be $t_0 = 2, t_1 = 3, t_2 = 12, t_3 = 10$.

1. Compute predictions for $\alpha = 0.2$ and $\alpha = 0.8$.
2. Which reacts faster to the phase change?
3. Compute absolute prediction error after each observation.
4. Explain how each predictor changes SJF/SRTF ordering.
5. Design a workload that causes repeated underprediction of a long job.

Exercise: exchange argument with weights

Suppose the objective becomes weighted completion time $\sum_i w_i c_i$.

1. Compare adjacent orders i, j and j, i .
2. Show that the preferred order depends on b_i/w_i rather than b_i alone.
3. Explain why plain SJF is no longer generally optimal.
4. Interpret a high weight operationally: urgency, cost, user share, or business value.

Further study

- weighted completion time, Smith's rule, and scheduling with release dates;
- non-clairvoyant scheduling and competitive analysis;
- multilevel feedback queues and burst classification;
- proportional-share, stride, and virtual-runtime scheduling;
- parallel-machine scheduling, affinity, and gang scheduling;
- deadline scheduling, admission control, and overload behavior.

Questions?